



Name:- Divya Waghmode

Class:- Sy-A

RollNo.:- A-28

Batch:- A2

Aim:- Data Visualization with Matplotlib: Data visualization on a dataset using matplotlib and seaborn libraries ● Scatter plot ● Line plot ● Bar plot ● Histogram ● Box plot ● Pair plot.

Assignment 5:-

1.What is the importance of data visualization in data analysis? How does visualization help in exploratory data analysis (EDA)?

Data visualization is an essential part of data analysis because it helps in presenting data in a graphical and easy-to-understand form. Instead of analyzing large tables of numbers, visualizations like graphs and charts allow users to quickly identify patterns, trends, and relationships in the data. In exploratory data analysis (EDA), visualization plays a key role in understanding the structure of the dataset, detecting outliers, identifying missing values, and discovering hidden patterns. It helps analysts make better decisions and choose appropriate methods for further analysis or modeling.

2.What is the difference between Matplotlib and other visualization libraries?

Ans:- Matplotlib is a basic and widely used plotting library in Python that provides complete control over creating graphs and charts. It is flexible but requires more code for customization. Other visualization libraries like Seaborn and Plotly offer higher-level interfaces and more attractive default styles. Seaborn is built on top of Matplotlib and provides better statistical visualization with less code, while Plotly allows interactive and dynamic graphs. In simple terms, Matplotlib is powerful and customizable, whereas other libraries are more user-friendly and visually appealing.

3.What is the purpose of the pyplot module in Matplotlib?

Ans:- The pyplot module in Matplotlib provides a simple and convenient way to create plots and graphs. It acts like a command interface that allows users to generate figures, plot data, and customize graphs using simple functions. With pyplot, users can easily create line plots, bar charts, histograms, and scatter plots without needing to manage complex details. It simplifies the plotting process and is

commonly used for quick and basic visualizations in Python.

4.Explain the process of creating a basic line plot.

Ans:- Creating a basic line plot involves a few simple steps using Matplotlib. First, the required library is imported. Then, data is prepared in the form of x-axis and y-axis values. After that, the plot() function is used to create the line graph by passing these values. Additional elements such as title, labels, and legends can be added for better understanding. Finally, the show() function is used to display the graph. This process helps in visualizing relationships between two variables in a simple and effective way.

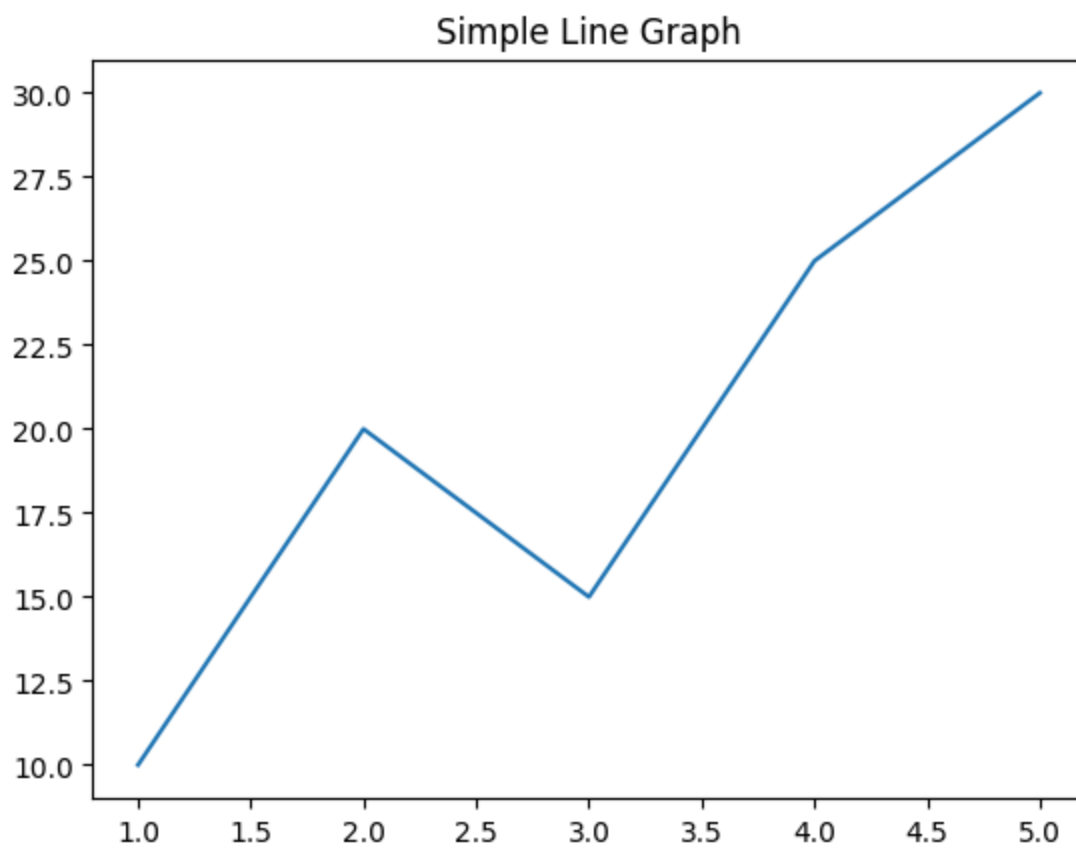
5.Why is customization important in data visualization? What customization options are available in Matplotlib?

Ans:- Customization is important in data visualization because it improves the clarity, readability, and presentation of graphs. Without proper customization, graphs may be difficult to understand or may not effectively communicate the intended message. In Matplotlib, various customization options are available such as changing colors, line styles, markers, titles, axis labels, legends, grid lines, and figure size. These options allow users to create more informative and visually appealing graphs, making it easier to interpret data and present it professionally.

```
In [ ]: import matplotlib.pyplot as plt
```

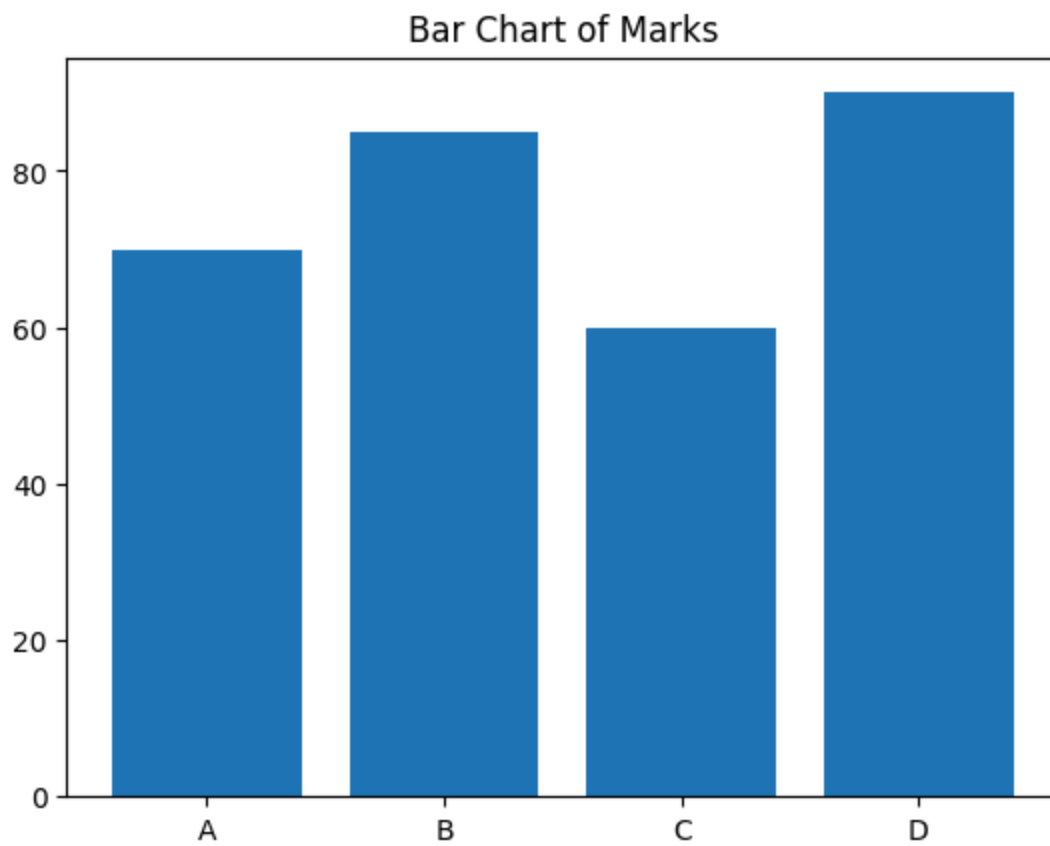
```
In [ ]: x = [1, 2, 3, 4, 5]
        y = [10, 20, 15, 25, 30]

        plt.plot(x, y)
        plt.title("Simple Line Graph")
        plt.show()
```



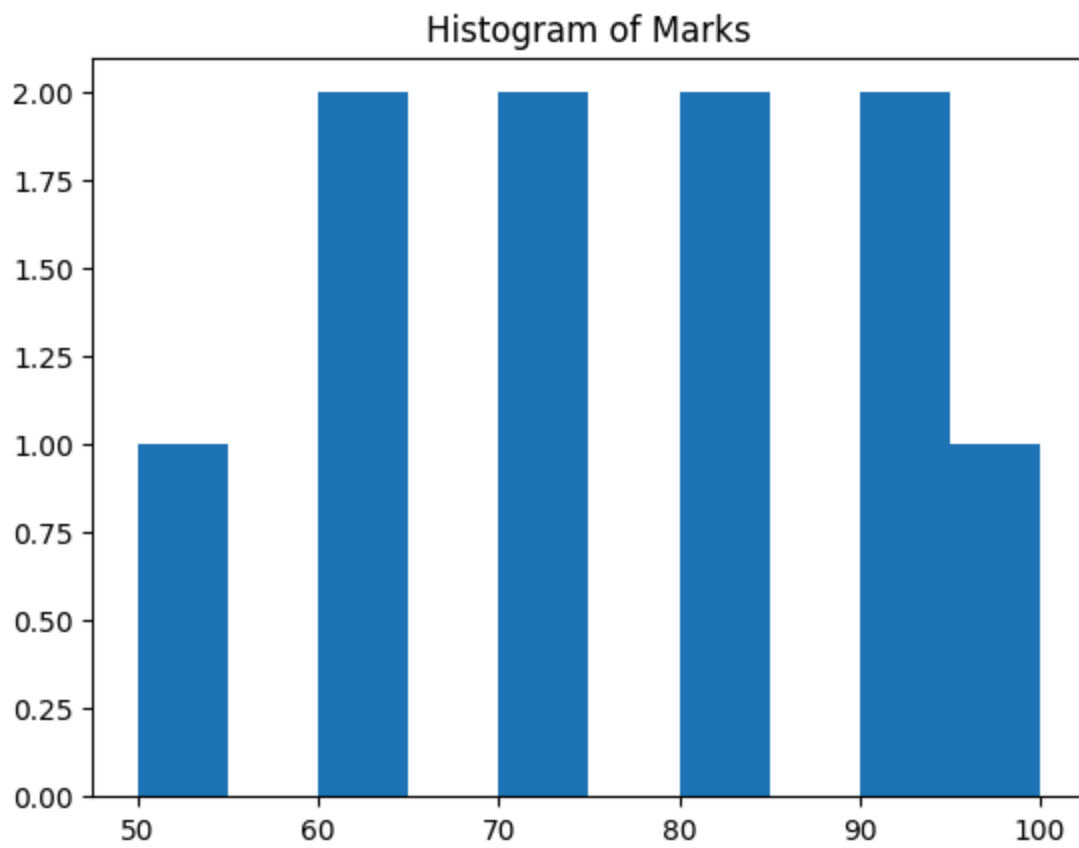
```
In [ ]: students = ["A", "B", "C", "D"]
marks = [70, 85, 60, 90]

plt.bar(students, marks)
plt.title("Bar Chart of Marks")
plt.show()
```



```
In [ ]: marks = [50, 60, 70, 80, 90, 60, 70, 80, 90, 100]

plt.hist(marks)
plt.title("Histogram of Marks")
plt.show()
```

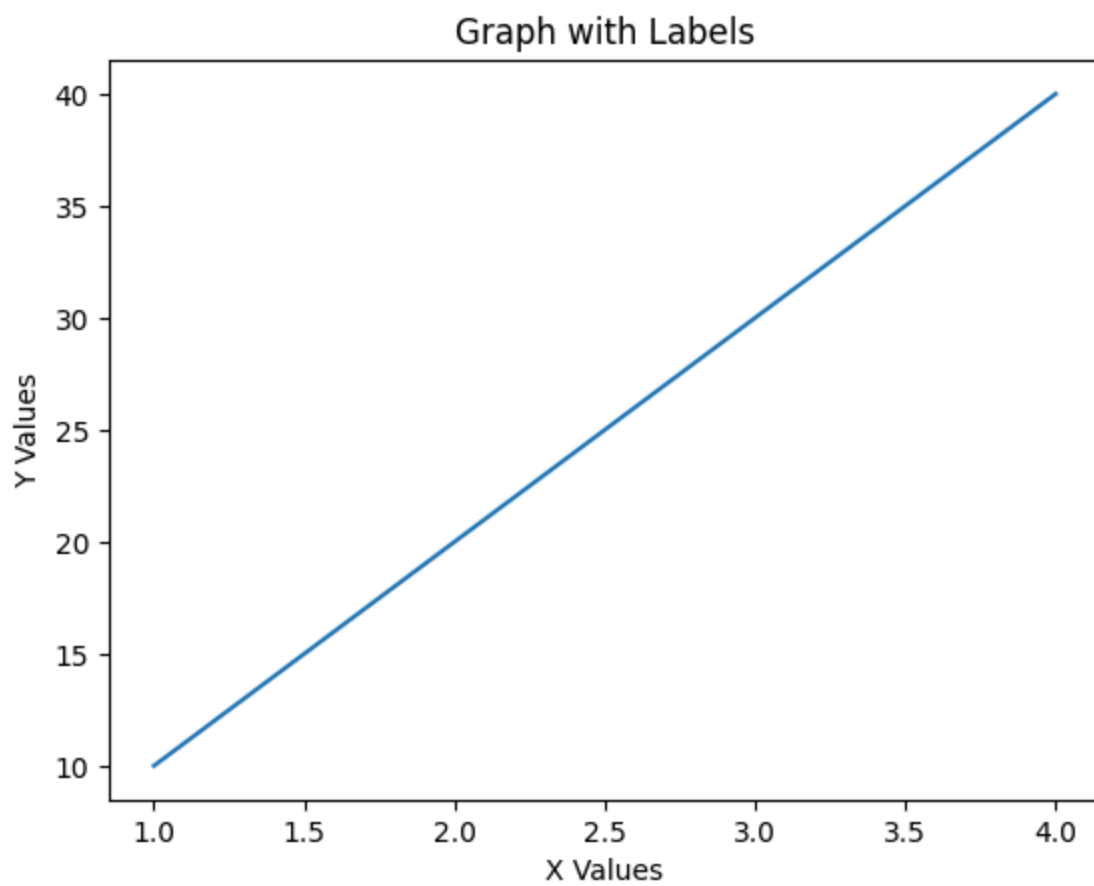


```
In [ ]: x = [1, 2, 3, 4]
        y = [10, 20, 30, 40]

        plt.plot(x, y)

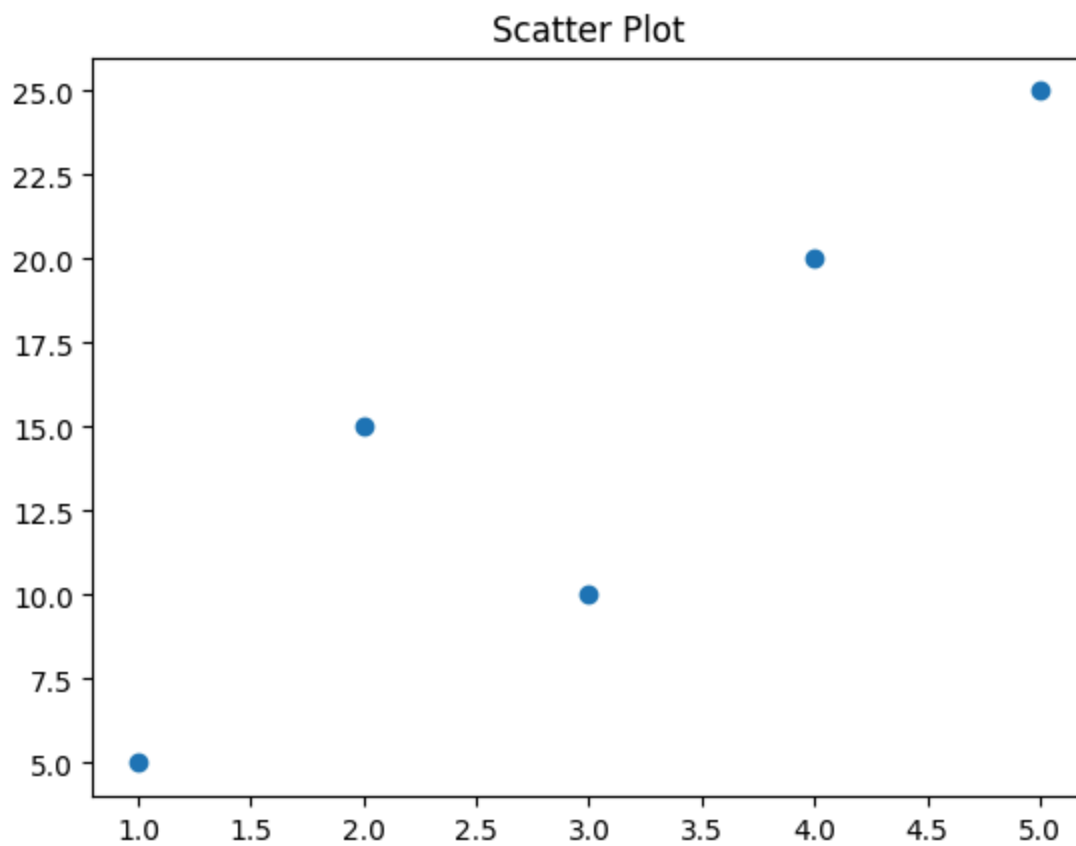
        plt.xlabel("X Values")
        plt.ylabel("Y Values")
        plt.title("Graph with Labels")

        plt.show()
```



```
In [ ]: x = [1, 2, 3, 4, 5]
        y = [5, 15, 10, 20, 25]

        plt.scatter(x, y)
        plt.title("Scatter Plot")
        plt.show()
```

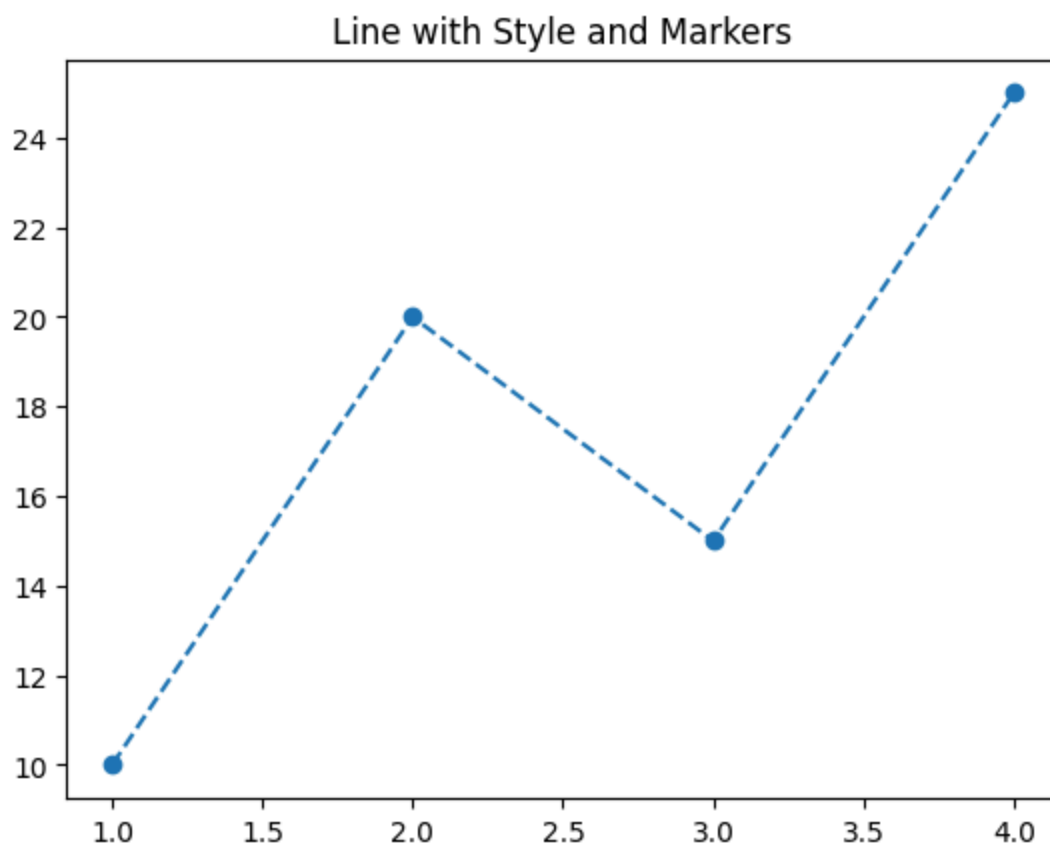


```
In [ ]: import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
```

```
In [ ]: x = [1, 2, 3, 4]
y = [10, 20, 15, 25]

plt.plot(x, y, linestyle='--', marker='o')

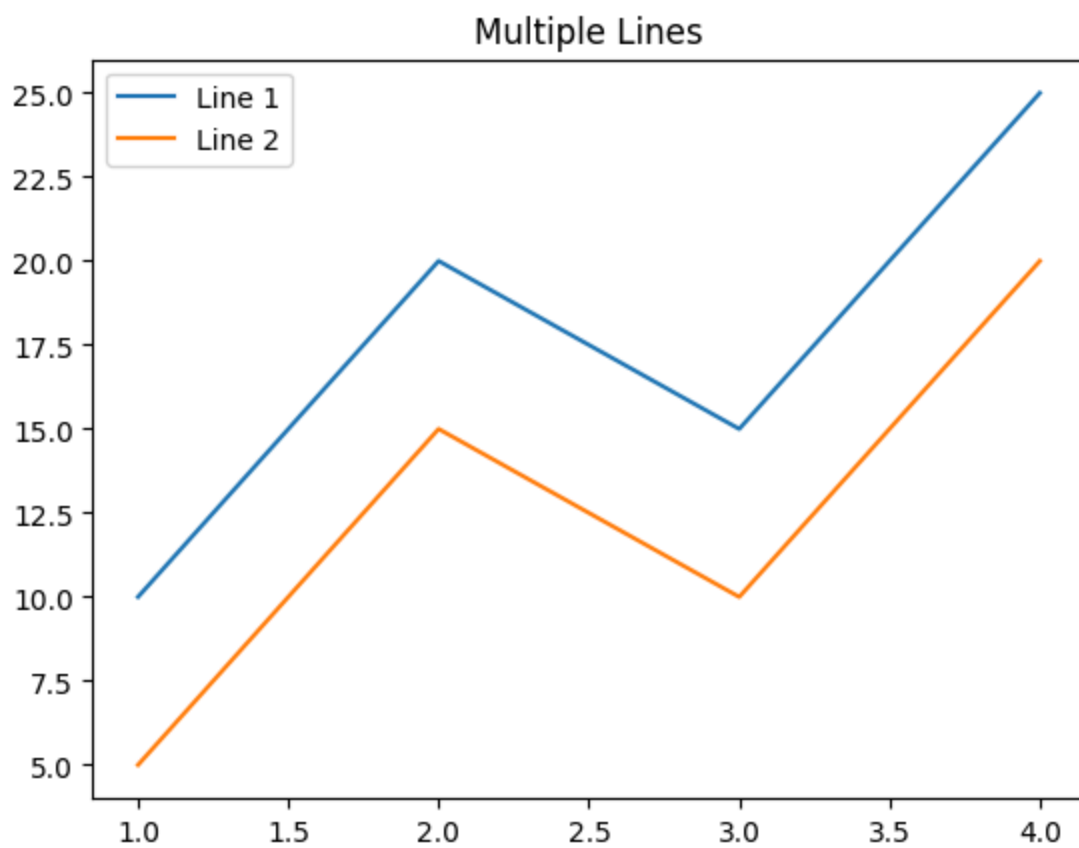
plt.title("Line with Style and Markers")
plt.show()
```



```
In [ ]: x = [1, 2, 3, 4]
        y1 = [10, 20, 15, 25]
        y2 = [5, 15, 10, 20]

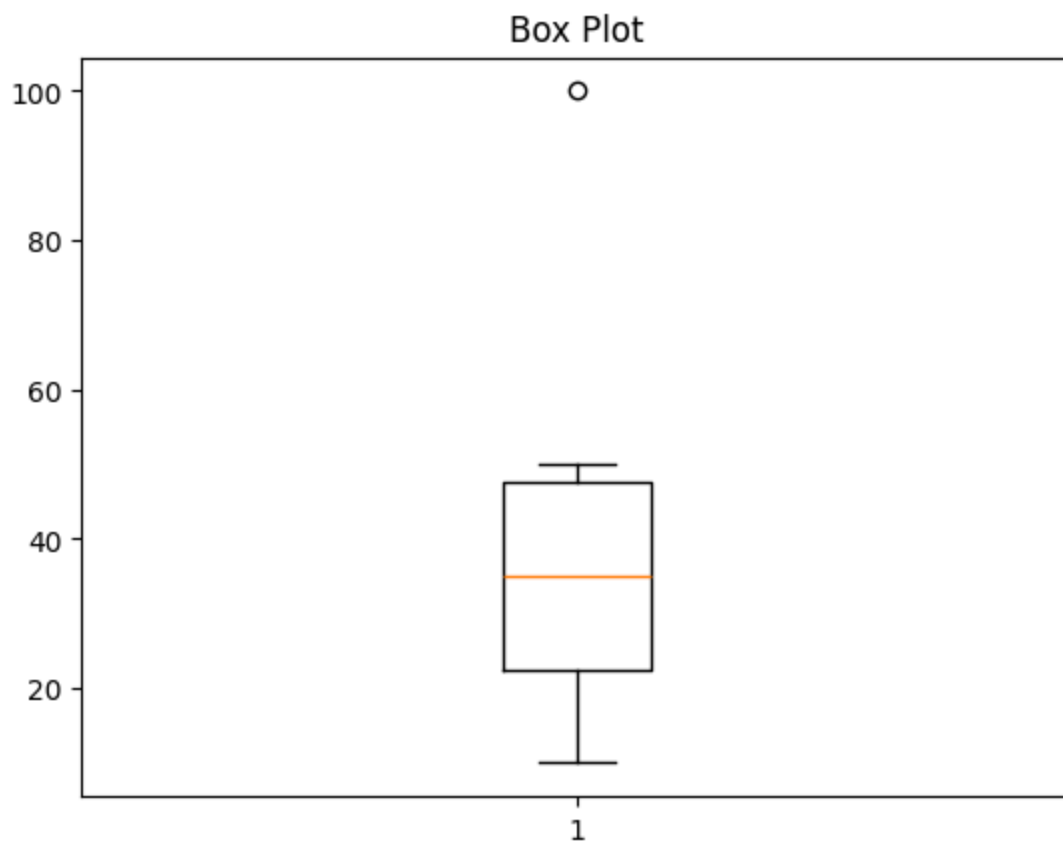
        plt.plot(x, y1, label="Line 1")
        plt.plot(x, y2, label="Line 2")

        plt.legend()
        plt.title("Multiple Lines")
        plt.show()
```



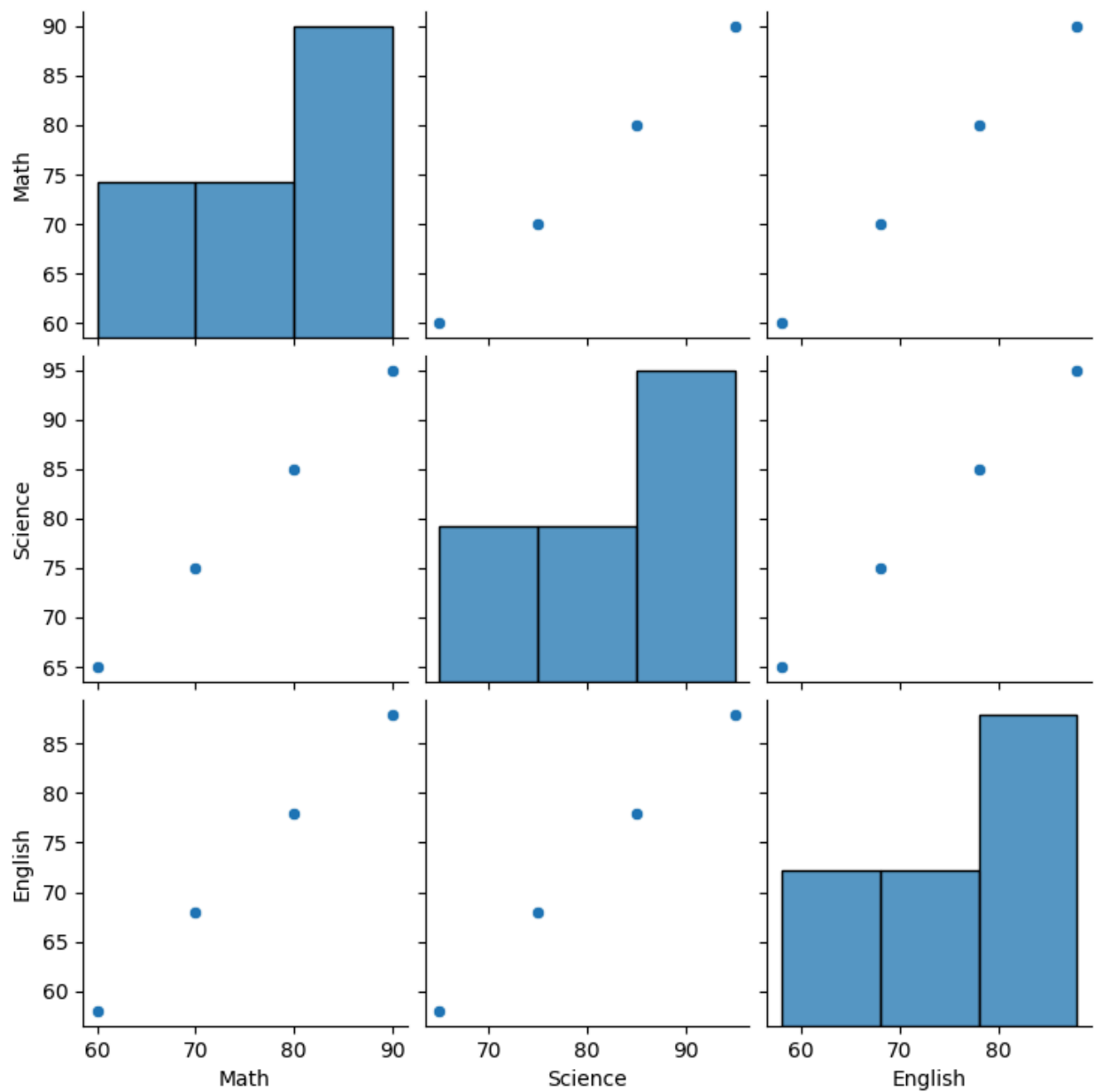
```
In [ ]: data = [10, 20, 30, 40, 50, 100]

plt.boxplot(data)
plt.title("Box Plot")
plt.show()
```



```
In [ ]: # Sample dataset
data = pd.DataFrame({
    "Math": [80, 70, 90, 60],
    "Science": [85, 75, 95, 65],
    "English": [78, 68, 88, 58]
})

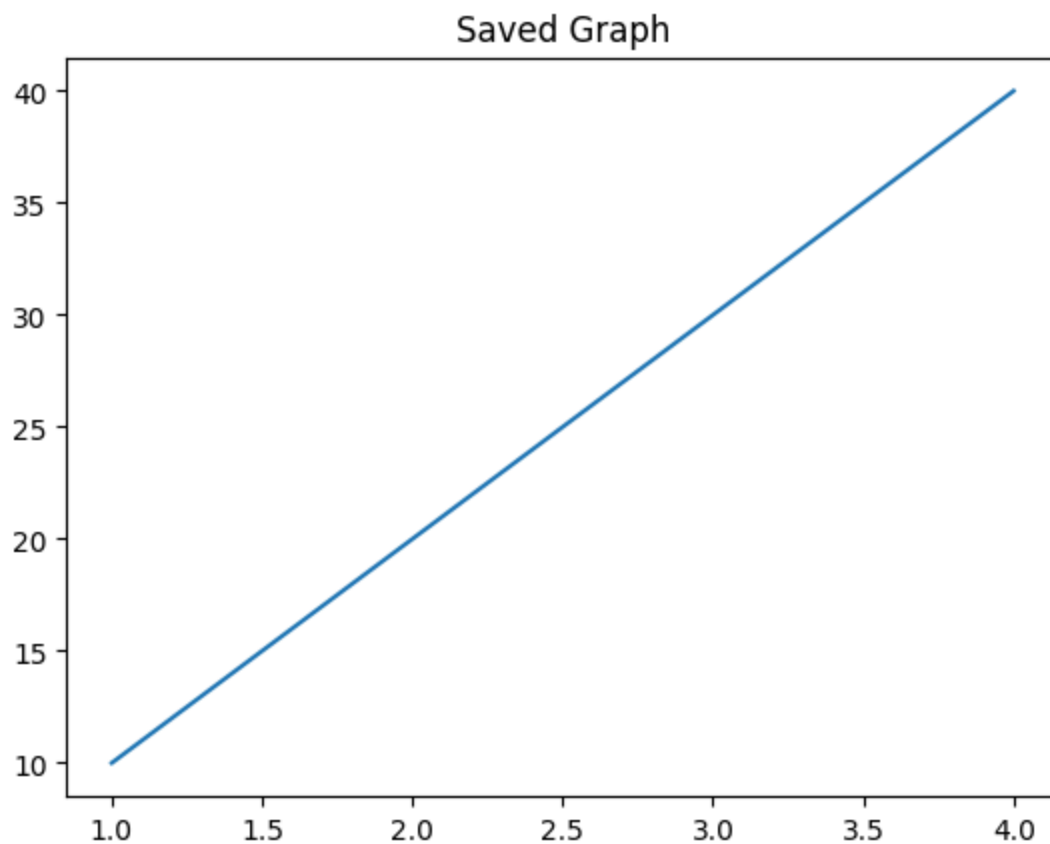
sns.pairplot(data)
plt.show()
```



```
In [ ]: x = [1, 2, 3, 4]
        y = [10, 20, 30, 40]

        plt.plot(x, y)
        plt.title("Saved Graph")

        plt.savefig("my_plot.png") # save image
        plt.show()
```



Conclusion:

This practical provided a clear understanding of data visualization techniques using the Matplotlib and Seaborn libraries. By creating different plots such as scatter plots, line plots, bar charts, histograms, box plots, and pair plots, we learned how to represent data in a graphical and meaningful way. These visualizations helped in identifying patterns, trends, and relationships within the dataset. Overall, the practical enhanced our ability to analyze and interpret data effectively using visual tools.